Model Context Protocol (MCP)

Open Standard for AI-Tool Integration

Anthropic (2024) | modelcontextprotocol.io | Linux Foundation Open Source

---

Introduction

The Model Context Protocol (MCP), released by Anthropic in November 2024, is an open standard that provides a universal way to connect AI models to external data sources and tools. MCP is now an open-source Linux Foundation project.

Source: modelcontextprotocol.io | anthropic.com/news/model-context-protocol

1. Why MCP?

Before MCP, every AI integration required custom code: Claude needed a bespoke connector for GitHub, a different one for Slack, another for PostgreSQL. MCP replaces this N×M integration problem with a single standard: any MCP client can talk to any MCP server.

2. Architecture

MCP uses a client-server architecture over JSON-RPC 2.0:

MCP Host: The AI application (e.g., Claude Desktop, Cursor, VS Code Copilot). Manages connections to multiple MCP servers simultaneously.

MCP Client: Protocol implementation inside the host. Maintains 1:1 connections with MCP servers. Handles capability negotiation and message framing.

MCP Server: Lightweight process exposing tools, resources, and prompts over stdio or HTTP/SSE. Examples: filesystem server, GitHub server, Postgres server.

Transport Layer: Two modes: stdio (local, subprocess communication) and HTTP+SSE (remote, production deployments with auth).

3. Three Core Primitives

Primitive	Controlled By	Description	Example
Tools	Model	Functions the LLM can call	create_file(), search_web()
Resources	Application	Data the app exposes as context	file://path, db://table

Prompts	User	Prompt templates surfaced in UI	/summarize, /review-pr

4. Building an MCP Server (Python)

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp import types

app = Server("my-knowledge-base")

@app.list_tools()
async def list_tools():
    return [
        types.Tool(
            name="search_kb",
            description="Search the internal knowledge base",
            inputSchema={
                "type": "object",
                "properties": {
                    "query": {"type": "string"},
                    "limit": {"type": "integer", "default": 5}
                },
                "required": ["query"]
            }
        )
    ]

@app.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "search_kb":
        results = your_vector_db.search(arguments["query"],
                                        k=arguments.get("limit", 5))
        return [types.TextContent(type="text", text=str(results))]

# Run over stdio (for Claude Desktop)
async def main():
    async with stdio_server() as streams:
        await app.run(*streams, app.create_initialization_options())

import asyncio
asyncio.run(main())
```

5. Claude Desktop Configuration

```
{
  "mcpServers": {
    "my-kb": {
```

```
      "command": "python",
      "args": ["/path/to/my_kb_server.py"]
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "ghp_..." }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres",
               "postgresql://user:pass@localhost/mydb"]
    }
  }
}
```

6. Official MCP Servers (Anthropic)

- filesystem — Read/write local files with configurable access controls
- github — Repository management, file operations, issue/PR creation
- google-drive — File search and read access to Google Drive
- slack — Channel/message read and write, user lookup
- postgres — Read-only SQL queries against PostgreSQL databases
- puppeteer — Browser automation and web scraping
- brave-search — Web and local search via Brave Search API
- memory — Persistent key-value store across conversations